Mini CPU em C

> Simulação de uma CPU mínima em linguagem C — fetch, decode, execute e contagem de números pares em memória.

Made with GAMMA

ARQUITETURA

Componentes da Mini CPU

mem[256]

256 bytes de memória endereçável. Dados e instruções compartilham o mesmo espaço.

reg[4]

4 registradores de uso geral (R0–R3), 8 bits cada.

PC / ZF

Program Counter aponta a próxima instrução. Zero Flag sinaliza resultado zero.

Ciclo de Execução

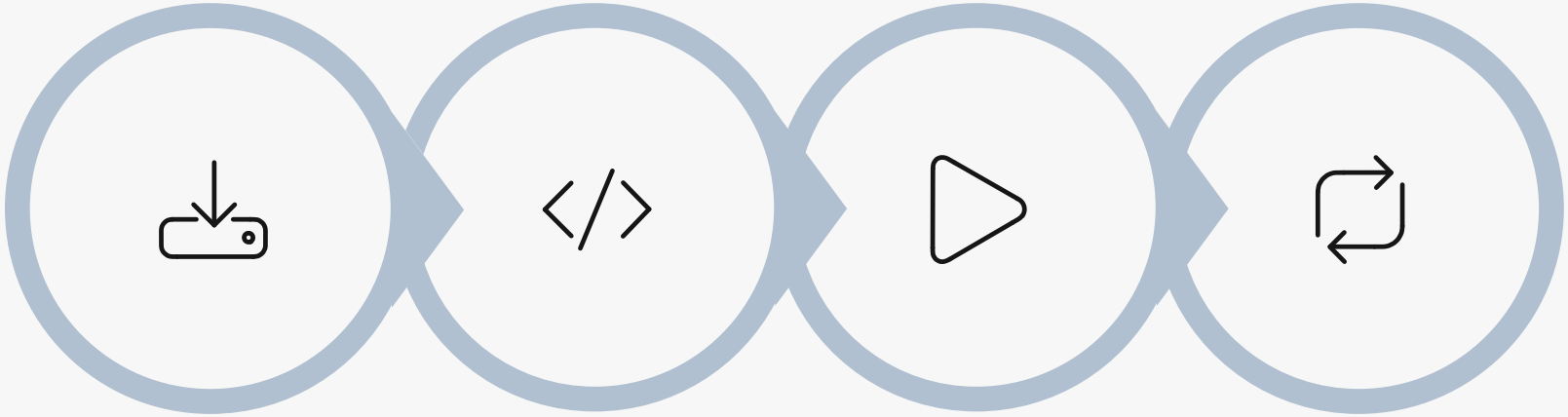
A CPU opera em ciclos discretos. Cada ciclo consome 3 bytes da memória: opcode, operando A e operando B.

```
ciclo = 0;
while (running) {
 fetch(&op, &a, &b);
 decode_execute(op, a, b);
 ciclo++;
}
```

Made with GAMMA

EXECUÇÃO

Ciclo Fetch → Decode → Execute



Fetch Decode Execute Trace

Cada instrução ocupa exatamente 3 bytes: `mem[PC]` = opcode, `mem[PC+1]` = operando A, `mem[PC+2]` = operando B. O PC avança 3 posições a cada ciclo.

Made with GAMMA

ISA

Conjunto de Instruções

0x01 LOAD

reg[a] = mem[b]

0x02 STORE

mem[b] = reg[a]

0x03 ADD

reg[a] += reg[b]

0x04 SUB

reg[a] -= reg[b]

0x05 MOV

reg[a] = b (imediato)

0x06 CMP

zf = (reg[a] == reg[b])

0x07 JMP

Salta incondicionalmente para endereço a

0x08 JZ

Salta se ZF == 1

0x09 JNZ

Salta se ZF == 0

0x0A HALT

Para a execução: running = 0

Made with GAMMA

PROBLEMA

Contar Números Pares na Memória

Mapa de Memória

```
Endereço | Conteúdo
---------+---------
0x10     | 4  (par)
0x11     | 7  (ímpar)
0x12     | 2  (par)
0x13     | 9  (ímpar)
0x14     | 6  (par)
0x15     | 1  (ímpar)
0x16     | 8  (par)
0x17     | 3  (ímpar)
...
0x20     | resultado
```

Objetivo

Percorrer os endereços `0x10` a `0x17`, identificar quais valores são pares e armazenar a contagem em `mem[0x20]`.

Restrição

Sem uso do operador `%`. A paridade é verificada por subtrações sucessivas de 2.

Resultado esperado

mem[0x20] = 4

Made with GAMMA

ALGORITMO

Estratégia de Verificação de Paridade

```
// Par: resíduo == 0
int valor = mem[0x10 + i];
while (valor >= 2) {
    valor -= 2;
}
if (valor == 0) pares++;
```

Subtrair 2 repetidamente equivale a calcular `valor % 2` sem usar o operador de módulo. Resultado `0` indica par; resultado `1` indica ímpar.

Made with GAMMA

CÓDIGO

Núcleo da CPU

```c
#include <stdio.h>
#include <stdint.h>

uint8_t mem[256] = {0};
uint8_t reg[4]  = {0};
uint8_t pc = 0, zf = 0;
uint8_t running = 1;
int ciclo = 0;

void fetch(uint8_t *op,
           uint8_t *a,
           uint8_t *b) {
  *op = mem[pc];
  *a  = mem[pc + 1];
  *b  = mem[pc + 2];
  pc += 3;
}

void decode_execute(uint8_t op,
                    uint8_t a,
                    uint8_t b) {
  switch (op) {
    case 0x01: reg[a] = mem[b];  break;
    case 0x02: mem[b] = reg[a];  break;
    case 0x03: reg[a] += reg[b]; break;
    case 0x04: reg[a] -= reg[b]; break;
    case 0x05: reg[a] = b;       break;
    case 0x06: zf = (reg[a]==reg[b]); break;
    case 0x07: pc = a;           break;
    case 0x08: if (zf)  pc = a;  break;
    case 0x09: if (!zf) pc = a;  break;
    case 0x0A: running = 0;      break;
  }
}
```

fetch()

Lê 3 bytes consecutivos da memória a partir do PC e avança o PC.

decode_execute()

Interpreta o opcode via `switch` e executa a operação correspondente nos registradores ou na memória.

ZF

Zero Flag atualizado pelo opcode `0x06` (CMP), usado pelos saltos condicionais JZ e JNZ.

Made with GAMMA

FUNÇÃO

Pré-carga e Contagem de Pares

```
void pre_carregar_e_contar_pares() {
  uint8_t valores[8] = {
    4, 7, 2, 9, 6, 1, 8, 3
  };

  // Carga na memória
  for (int i = 0; i < 8; i++) {
    mem[0x10 + i] = valores[i];
  }

  // Contagem sem operador %
  int pares = 0;
  for (int i = 0; i < 8; i++) {
    int valor = mem[0x10 + i];
    while (valor >= 2) {
      valor -= 2;
    }
    if (valor == 0) {
      pares++;
    }
  }

  mem[0x20] = pares;
}
```

Fluxo linha a linha

1 Inicializa mem[0x10..0x17]

8 valores carregados diretamente no vetor de memória.

2 Loop de paridade

Para cada byte, subtrai 2 até o valor ser menor que 2.

3 Armazena resultado

`mem[0x20] = pares` — saída do programa.

Made with GAMMA

RESULTADO

Estado Final da Memória

Dump de memória

```
Addr  Val  Paridade
----  ---  --------
0x10  4    par  *
0x11  7    ímpar
0x12  2    par  *
0x13  9    ímpar
0x14  6    par  *
0x15  1    ímpar
0x16  8    par  *
0x17  3    ímpar
...
0x20  4    <-- resultado
```

4

Números pares

4, 2, 6, 8 — identificados por resíduo zero.

4

Números ímpares

7, 9, 1, 3 — descartados da contagem.

O valor `4` é gravado em `mem[0x20]` ao final da execução da função.

Made with GAMMA

CONCLUSÃO

Obrigado pela Atenção!

```
#include <stdio.h>

int main() {
    printf("Obrigado pela atenção!\n");
    return 0;
}
```

Made with GAMMA